

Reviewer Pack — Minimal Rank of Quadratic Forms Bounds Amplitude Amplificati...

Entry e5f43b1da914 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 10:42:01 UTC

Minimal Rank of Quadratic Forms Bounds Amplitude Amplification in Quantum Computing

Entry ID: e5f43b1da914 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 10:42:01 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quadratic Forms) **Field B** (complexity object): Quantum Computation: Amplitude Amplification

Statement:

{‘st1’: ‘For a quantum algorithm A , let $Q(A)$ be the set of quadratic forms that describe the error in its amplitude amplification process. We conjecture that for all quantum algorithms A with amplitude amplification factor α , the minimal rank of any form in $Q(A)$ is bounded above by $O(\log(\alpha))$ ’, ‘st2’: ‘This implies that the efficiency of amplitude amplification in quantum computing can be characterized by the geometric properties of the corresponding quadratic forms.’, ‘st3’: ‘Furthermore, we propose that for a given α , there exists an algorithm that can find a form in $Q(A)$ with rank at most $O(\log(\alpha))$ ’}

Rationale (proposer’s reasoning):

{‘r1’: ‘Quadratic forms play a crucial role in understanding the geometry of quantum states and their transformations.’, ‘r2’: ‘The minimal rank of these forms could potentially reveal hidden structure within quantum algorithms, specifically those involving amplitude amplification.’, ‘r3’: ‘If this conjecture holds, it would provide a new perspective on optimizing quantum algorithms by focusing on the algebraic properties of their error descriptions.’}

Taxonomy category: EXAMPLE_1 (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 661a5bc8061c5fa1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated quantum circuits, the minimal rank of the quadratic forms describing amplitude amplification error is less than or equal to $O(\log(\alpha))$ with a mean of the ranks across 30 seeds not exceeding 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'algebraic geometry' AND 'quadratic forms' AND 'amplitude amplification' AND 'quantum computing' - 'minimal rank' AND 'quadratic forms' AND 'bound by log(alpha)' AND 'quantum algorithms' - 'error in quantum algorithms' AND 'described by quadratic forms' AND 'amplitude amplification efficiency'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rref = [row[:] for row in matrix]

    for i in range(rows):
        # Find pivot
        max_row = i
        for r in range(i+1, rows):
            if abs(rref[r][i]) > abs(rref[max_row][i]):
                max_row = r

        # Swap rows
        rref[i], rref[max_row] = rref[max_row], rref[i]

        # Eliminate below pivot
        for r in range(i+1, rows):
            factor = Fraction(-rref[r][i], rref[i][i])
            for c in range(cols):
                rref[r][c] += factor * rref[i][c]

    return rref

def rank(matrix):
    rref = gaussian_elimination(matrix)
    rank = 0
```

```

    for row in rref:
        if any(row):
            rank += 1
    return rank

def generate_quadratic_form(n, seed):
    random.seed(seed)
    Q = [[random.uniform(-1, 1) for _ in range(n)] for _ in range(n)]
    Q = [row[:] for row in Q] # Ensure Q is copied to avoid modifying the original
    return Q

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_rank = 0
    instances_tested = 0

    for n in n_values:
        Q = generate_quadratic_form(n, seed)
        min_rank = rank(Q)
        total_rank += min_rank
        instances_tested += 1

    mean_rank = total_rank / len(n_values)

    if mean_rank <= 3 * math.log(2): # Assuming  $\alpha = 2$  for simplicity
        conjecture_holds = True
        counterexample = ""
    else:
        conjecture_holds = False
        counterexample = "Mean rank exceeds  $O(\log(\alpha))$ "

    return {
        "metric_name": "mean_rank",
        "metric_value": mean_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or list(range(2, 53)) # Default to first 30 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=... support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):

```

```

        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture"])
        print(f"RESULT: FALSIFIED counterexample=\"Mean rank exceeds 0(log(α))\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_20767162.py", line 89, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_20767162.py", line 62, in run_trial
    min_rank = rank(Q)
                ^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_20767162.py", line 41, in rank
    rref = gaussian_elimination(matrix)
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_20767162.py", line 34, in gaussian_elimination
    factor = Fraction(-rref[r][i], rref[i][i])
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate the cause of the crash and rerun the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11946
2	preregistration	ollama_remote	glm4:latest	0	0	5454
3	novelty	ollama_remote	glm4:latest	0	0	4996
4	novelty	ollama_remote	glm4:latest	0	0	5400

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15987
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9884
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9415
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9422
9	judge	ollama_remote	glm4:latest	0	0	10572

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 83077 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e5f43b1da914.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e5f43b1da914.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e5f43b1da914.tar.gz` (if generated)